# Technical Supplement to Adaptive HACT: Information Limits and Safe Adaptation of Verification Evidence

Da Xu and Fei Wang
China Mobile Research Institute, Beijing, China
Corresponding author: xudayj@chinamobile.com

This supplement accompanies the primary manuscript. It supplies the full complete-epoch orbit certificate, fixed-width padding proof and a practical route through the reproducibility artifact. It is supporting material for one submission. All numerical observations retain their original provenance; the v15 additions are separate compiler replays and CPU diagnostics, not new checker executions, consumer studies or agent evaluations.

## S1 Notation and scope

A rooted tree represents a registry of $n$ distinct checks. Internal arities lie in $\{2, \dots, b\}$, and a packet costs $c(d) = H + Sd$ bytes. A node with semantic coverage $G$ is emitted once for each completion wave that intersects $G$. A strict epoch additionally constructs every internal packet once. Thus $W(T) = \sum_v c(d_v)$ is construction cost and $h_e(G)$ counts touched waves. Acceptance still requires every current approved check to pass; none of these costs changes that predicate. The main article gives the parameter-general persistent-refresh separation and the global exchangeable optimum. The following two proofs complete its complete-epoch and fixed-width extensions.

## S2 Complete-epoch separation and orbit certificate

### S2.1 The separation also occurs in complete epochs

The preceding construction uses persistent refresh events. A complete verification epoch imposes more structure: the completion sets are disjoint and cover every declared check. That restriction does not make pairwise summaries sufficient.

**Theorem S1** (Complete-epoch information separation)**.** *For any integer fanout $b \geq 6$, $S > 0$ and $16/11 < H/S < 28/17$, there are two laws on six checks partitioned into exactly three nonempty refresh windows, with one full initial construction, that have identical population singleton and pairwise completion statistics but disjoint unique global tree optima. Writing*

$$\Delta_P = (28S - 17H)/8, \qquad \Delta_Q = (11H - 16S)/5,$$

*the sharp deterministic pairwise-only regret is $\min(\Delta_P, \Delta_Q)$; the sharp randomized minimax regret is $\Delta_P \Delta_Q/(\Delta_P + \Delta_Q)$.*

*Proof.* Let $G = \{0, 1, 2, 3\}$ and $O = \{4, 5\}$. A block's type $(g, o)$ counts its members from $G$ and $O$. Let $A = (G, \{4\}, \{5\})$, and let $B, C, D$ be uniform laws over all partitions of the respective block-type multisets

$$B : \{(2,1), (2,0), (0,1)\},$$
$$C : \{(2,1), (1,1), (1,0)\},$$
$$D : \{(2,0), (1,1), (1,1)\}.$$

Table 1: Complete-epoch orbit certificate after contracting arities two and three. Entries are total expected bytes divided by $S$, with $x = H/S$. The flat comparator is $4x + 24$.

| Child type | $P$ | $Q$ |
|---|---|---|
| $(2,2)$ | $\frac{383}{48}x + \frac{335}{12}$ | $\frac{239}{30}x + \frac{418}{15}$ |
| $(3,1)$ | $\frac{113}{16}x + \frac{97}{4}$ | $\frac{1131}{160}x + \frac{971}{40}$ |
| $(3,2)$ | $8x + 28$ | $\frac{639}{80}x + \frac{447}{16}$ |
| $(4,0)$ | $\frac{49}{8}x + \frac{41}{2}$ | $\frac{31}{5}x + \frac{104}{5}$ |
| $(4,1)$ | $\frac{113}{16}x + \frac{373}{16}$ | $\frac{71}{10}x + \frac{47}{2}$ |

There are 12, 24 and 12 partitions in those families. Define

$$P = \tfrac{7}{8}A + \tfrac{1}{8}B, \qquad Q = \tfrac{9}{10}A + \tfrac{1}{20}C + \tfrac{1}{20}D.$$

Every check completes once in every outcome. Each within-$G$ pair shares a window with probability $11/12$, each $G$–$O$ pair with probability $1/32$, and the $O$ pair with probability zero, under either law. Uniformly permuting each partition's three windows also makes every window-specific first and second moment agree. The expected number of windows touching $G$ is nevertheless $9/8$ under $P$ and $6/5$ under $Q$.

We next reduce *all* tree shapes and leaf permutations. A node's weight, including its initial packet, lies in $[2, 4]$. Contracting a nonroot internal child of arity $d = 2$ or 3 raises the parent's price by $S(d-1)$ per emission and removes that child's packet. Its cost change in every complete three-window outcome is at most

$$4S(d-1) - 2(H + Sd),$$

which is $-2H$ for $d = 2$ and $-2(H - S)$ for $d = 3$, both negative. Contraction stays feasible because there are only six leaves and $b \geq 6$. Since $\sum_v (d_v - 1) = 5$, a tree with at least three internal nodes must contain such a nonroot child. Repeated contractions leave a flat root or one root with a single internal child. Arity-two/three children are themselves dominated by the flat root.

The remaining five child types are listed in Table 1, with $x = H/S$. Direct counting in $A, B, C, D$ gives each entry. Every row except $(4,0)$ exceeds the flat cost $4x + 24$ throughout the stated interval. The $(4,0)$ tree has root children $G, \{4\}, \{5\}$; its $P$ and $Q$ costs are respectively

$$J_P(T_G) = \tfrac{49}{8}H + \tfrac{41}{2}S, \qquad J_Q(T_G) = \tfrac{31}{5}H + \tfrac{104}{5}S.$$

The flat tree costs $4H + 24S$ in both worlds. Thus $T_G$ is uniquely preferred under $P$ and flat is uniquely preferred under $Q$, with gaps $\Delta_P, \Delta_Q > 0$.

For the minimax statement, larger trees cannot improve either action: each contraction saves at least $m = 2(H - S)$, and $m - \Delta_P = (33H - 44S)/8 > 0$ in the interval. Even a tree contracting to $T_G$ is therefore componentwise dominated by flat. The only nondominated actions are $T_G$ and flat. Choosing $T_G$ with probability $p$ has regrets $(1-p)\Delta_P$ and $p\Delta_Q$; equating them proves the randomized formula, attained at $p = \Delta_P/(\Delta_P + \Delta_Q)$. □

At the artifact's $H = 512, S = 320$, $T_G$ costs 9,696 bytes under $P$ and $49,152/5 = 9,830.4$ under $Q$; flat costs 9,728. The deterministic penalty is 32 bytes and randomized minimax regret is $512/21 \simeq 24.381$ bytes, attained with group probability $5/21$. An independent subset reference and separately written literal evaluator agree on all 2,752 labeled unordered six-leaf trees, including uniqueness. An exact affine certificate checks all two-node orbit classes across the whole price interval. An integration test also fits the implemented full-wave refiner on both exact mixtures and verifies its two different optima using independent packet pricing; this is analytical conformance, not workload performance evidence.

This is a constructed complete-epoch result, not an observed software improvement. It allows three externally chosen refresh windows with variable membership and width. The six-check CLI with its fixed eight-record flush would instead produce one wave, so it does not instantiate this witness. Other higher-order summaries, including wave-size distributions, may distinguish these laws; the lower bound concerns the specified pairwise-only information interface.

| Orbit | Wave 1 | Wave 2 | Wave 3 |
|---|---|---|---|
| $A$ | $\{0,1,2,3\}$ | $\{4\}$ | $\{5\}$ |
| $B$ | $\{0,1,4\}$ | $\{2,3\}$ | $\{5\}$ |
| $C$ | $\{0,1,4\}$ | $\{2,5\}$ | $\{3\}$ |
| $D$ | $\{0,1\}$ | $\{2,4\}$ | $\{3,5\}$ |

Figure 1: Representative six-check partitions in Theorem 3. Here $G = \{0,1,2,3\}$ and $O = \{4,5\}$. Each orbit is uniform over all renamings within $G, O$; wave order is uniformly permuted. $P = 7A/8 + B/8$ and $Q = 9A/10 + C/20 + D/20$ share the stated pairwise completion laws. These are complete epochs, unlike the main article's persistent partial-refresh construction.

# S3 Fixed-width padding and rank equivalence

## S3.1 Fixed-width batches retain a local information gap

The previous theorem allows variable window widths. The following lift connects the information limit to the evaluated eight-completion flush. Its conclusion concerns two fixed feasible layouts; it does not assert global optimality over all trees of the enlarged registry.

**Lemma S2** (Uniform equal-width padding)**.** *Let a random partition of $n$ original checks have $q$ blocks of size at most $w$. Add $F = qw - n \geq 2$ distinct filler checks, uniformly allocating them to the block deficits. If $p_{ij}$ is the original probability that $i, j$ share a block, then the same-block probabilities are*

$$p_{if} = \frac{w - 1 - \sum_{j \neq i} p_{ij}}{F}, \tag{1}$$

$$p_{fg} = \frac{qw(w-1) - (2w-2)n + 2\sum_{i<j} p_{ij}}{F(F-1)} \quad (f \neq g). \tag{2}$$

*Thus identical original pairwise probabilities imply identical pairwise probabilities on the padded registry.*

*Proof.* If $i$ lies in a block of size $k$, its block receives $w - k$ fillers, so a named filler joins it with probability $(w-k)/F$. Taking expectations and using $\mathbb{E}(k-1) = \sum_{j \neq i} p_{ij}$ gives (1). For distinct fillers the conditional probability is $\sum_{a=1}^{q} (w - k_a)(w - k_a - 1)/[F(F-1)]$. Expand the numerator and use $\sum_a k_a = n$ and $\mathbb{E}\sum_a k_a(k_a - 1) = 2\sum_{i<j} p_{ij}$ to obtain (2). Original pair memberships are unchanged. □

**Proposition S3** (Fixed-eight candidate-class separation)**.** *For $b \geq 6$ and the price interval of Theorem S1, there are two laws on complete 24-check execution sequences with identical population bivariate completion-rank distributions, but opposite preferred choices in a two-layout candidate class under consecutive batches of eight. The two wrong-choice penalties remain $\Delta_P, \Delta_Q$; the class-relative sharp randomized minimax regret is $\Delta_P \Delta_Q / (\Delta_P + \Delta_Q)$.*

*Proof.* Apply Lemma S2 to the six-check laws with $q = 3, w = 8, F = 18$. Their largest original block has size four. Original–filler probabilities are $67/288$ for $G$ and $55/144$ for $O$; filler–filler probability is $191/612$, in both worlds. Uniformly permute the three blocks and independently shuffle the eight members of each. For two distinct checks with same-block probability $p$, the probability of any specified pair of distinct completion ranks is $p/[qw(w-1)]$ if the ranks lie in one batch, and $(1-p)/[q(q-1)w^2]$ otherwise. Singleton ranks are uniform. Hence every bivariate rank distribution agrees, while grouping consecutive ranks by eight recovers the padded partition exactly.

Place the original six checks below one child of a common root and a fixed feasible filler-only tree below its other child. The two candidate layouts use respectively the flat six-leaf subtree and $T_G$ from Theorem S1. The root and filler subtree contribute the same cost to both choices within each world, including construction. The original subtree's touched windows are unchanged. Subtracting the common terms preserves $\Delta_P$ and $\Delta_Q$, and the same two-action minimax calculation applies. Common costs need not agree between worlds. □

At the implemented prices, the wrong-choice penalties are 32 and 102.4 bytes and the class-relative randomized penalty is 512/21 bytes. The exact reference enumerates all 740 and 2,540 positive-mass padded partitions in the smaller $w = 4, F = 6$ instance, independently checks the pairwise formulas, and checks actual batching and packet costs on 600 complete 24-check sequences. Thirty tests cover these identities and deliberate corruptions. Enumeration and representative sequence conformance support the implementation; the padding and cancellation arguments establish the general claim. Neither this construction nor its rank equivalence implies that empirical pairwise-summary vectors have identical joint laws, or that the two embedded layouts are globally optimal on 24 checks.

# S4 Exact scoring and conditional adoption

## S4.1 Conditional adoption with lifecycle costs

Selecting the smallest observed cost does not establish a net deployment benefit. The following rule makes the additional assumptions explicit. It specializes standard bounded-loss calibration and multiple-comparison control [1, 2]; it introduces no new concentration inequality. Its HACT-specific step is to bound *paired* differences after cancelling packets with the same semantic coverage.

Fix an incumbent $L_0$ and $K \geq 1$ training-fitted candidates $L_j$ before calibration. All layouts use the same registry and packet contract. An epoch partitions the $n$ checks into exactly $q$ nonempty waves of at most $w$ checks, with $q \leq n \leq qw$. Let $h_e(G)$ count the waves that intersect semantic coverage $G$. Aggregate the internal packet prices by coverage and subtract the candidate's coefficients from the incumbent's:

$$a_{jG} = \sum_{v:G_{L_0}(v)=G} c(d_v) - \sum_{v:G_{L_j}(v)=G} c(d_v).$$

Coefficients may have either sign. Shared packets cancel, regardless of their display positions. The strict-epoch saving is exactly

$$\begin{aligned} d_j(e) &= \ell_e(L_0) - \ell_e(L_j) \\ &= a_{j0} + \sum_G a_{jG} h_e(G), \quad a_{j0} = \sum_G a_{jG}. \end{aligned} \tag{3}$$

For $s = |G|$, the support implies

$$l_G = \max\{\lceil s/w \rceil, q - (n - s)\} \leq h_e(G) \leq u_G = \min(s, q). \tag{4}$$

The second lower bound follows because every untouched nonempty wave requires a distinct check outside $G$. In particular $h_e(\mathcal{R}) = q$. Define

$$\begin{aligned} A_j &= a_{j0} + \sum_{a_{jG}>0} a_{jG} l_G + \sum_{a_{jG}<0} a_{jG} u_G, \\ B_j &= a_{j0} + \sum_{a_{jG}>0} a_{jG} u_G + \sum_{a_{jG}<0} a_{jG} l_G. \end{aligned}$$

Set $R_j = B_j - A_j$. Thus $d_j(e) \in [A_j, B_j]$ on the entire declared support, rather than merely the observed epochs. Bounds for different coverages need not be jointly attainable, so this envelope is conservative.

Pairwise insufficiency does not require retaining every raw wave forever. Once a catalogue is frozen, exact paired scoring needs only the accumulated sum of each $d_j(e)$ and the number of epochs. More generally, the following standard linear-identification argument characterizes population scoring interfaces, and distinguishes exact score recovery from argmin recovery.

**Proposition S4** (Exact population score identification)**.** *On the finite epoch support $\Omega$, let an interface expose only $\mathbb{E}_P\phi_1, \ldots, \mathbb{E}_P\phi_p$ for all probability laws $P$. It determines every expected paired score $\mathbb{E}_P d_j$, for every $P$, if and only if each function $d_j$ belongs to* $\operatorname{span}\{1, \phi_1, \ldots, \phi_p\}$ *on $\Omega$. The smallest number of scalar expectation features for exact scoring is the dimension of the span of the $d_j$ modulo constant functions.*

*Proof.* Span membership supplies an affine recovery formula. Conversely, if $d_j$ is outside the span, finite-dimensional orthogonality supplies a signed vector $z$ annihilating 1 and every $\phi_i$ but not $d_j$. Its positive and negative parts have the same nonzero mass. Normalizing them yields laws $P, Q$ with equal exposed summaries but different expected paired scores. The dimension statement follows by taking a basis modulo constants and applying the same argument to any lower-dimensional feature span. This concerns exact scoring; different scores alone do not imply different minimizing layouts, which requires the earlier separations. □

The implementation accumulates exact integer paired sums in one pass with $O(K)$ score counters, in addition to the frozen coverage coefficients and one epoch's masks. It retains no per-epoch loss rows on this path. Auditable source/checker records and any required raw provenance remain separate; score sufficiency grants no permission to discard acceptance evidence.

**Theorem S5** (Conditional expected lifecycle benefit)**.** *Condition on training and the frozen catalogue, support, $0 < \delta < 1$, TV radius $0 \le \rho \le 1$, horizon $h \ge 1$ and nonnegative installation costs $I_j$, all specified before calibration. Suppose the $m \ge 1$ calibration epochs are IID from $P$ and independent of fitting. Let $\bar{d}_j$ be their paired mean, and define*

$$t_j = R_j \sqrt{\frac{\log(K/\delta)}{2m}},$$
$$g_j = \max\{A_j, \bar{d}_j - t_j - \rho R_j\} - I_j/h. \tag{5}$$

*With probability at least $1 - \delta$ over calibration, simultaneously for all candidates and every future law $Q$ on the same support with $\mathrm{TV}(P, Q) \le \rho$,*

$$h\,\mathbb{E}_Q d_j - I_j \ge h g_j.$$

*Adopting only a candidate with $g_j > 0$, otherwise retaining $L_0$, therefore certifies a conditional positive expected padded-byte benefit. This does not certify realized savings, elapsed time, or task acceptance.*

*Proof.* For each fixed candidate the one-sided Hoeffding inequality gives $\Pr\{\mathbb{E}_P d_j < \bar{d}_j - t_j\} \le \delta/K$ when $R_j > 0$. When $R_j = 0$ the cost difference is constant and the conclusion holds deterministically. A union bound makes these inequalities simultaneous; candidate differences within an epoch need not be independent.

Use the convention $\mathrm{TV}(P, Q) = \sup_E |P(E) - Q(E)|$. For $R_j > 0$, normalize $(d_j - A_j)/R_j$ to $[0, 1]$ and integrate its superlevel-set probabilities. It follows that $|\mathbb{E}_Q d_j - \mathbb{E}_P d_j| \le \rho R_j$, uniformly over the TV ball. For $R_j = 0$ the difference is zero. Since $Q$ remains on the same support, also $\mathbb{E}_Q d_j \ge A_j$. Combining the bounds and subtracting $I_j/h$ proves the claim. The simultaneous event permits selection of any qualifying candidate after calibration. □

The rule conditions on a *justified* sampling design and shift radius; neither is learned or verified from repeated observations by this theorem. If deployment can leave the declared support, the guarantee does not apply. Reusing calibration to refit layouts, choose the radius, or repeatedly deploy new catalogues needs additional error accounting. The implementation binds all plan parameters to a digest and requires external sampling/freeze provenance; an assertion string alone is not evidence of independence or chronology. Missing provenance or radius returns UNKNOWN and retains the incumbent. Installation costs are bytes in the declared protocol; compilation seconds and consumer utility remain separate admission tests. An installation operation already charged inside the first epoch must not also appear in $I_j$; the cost ledger must assign each operation once.

# S5 Calibration horizons and variance-sensitive alternatives

The calibration example is a stress test of a conditional certificate, not a recommendation to execute 100,000 suites before changing a layout. The main article's synthetic adoption table samples dependent prefixes within each seed; its counts do not estimate the minimum required sample size.

For a frozen catalogue and $m \ge 2$ IID epochs, a standard empirical-Bernstein bound [3], applied to the negated and range-normalized paired saving, replaces the Hoeffding radius by

$$t_j^{\mathrm{EB}} = \sqrt{\frac{2\widehat{v}_j \log(2K/\delta)}{m}} + \frac{7R_j \log(2K/\delta)}{3(m-1)},$$

where $\widehat{v}_j$ is the unbiased sample variance in squared bytes. Using $\max\{A_j, \bar{d}_j - t_j^{\mathrm{EB}} - \rho R_j\} - I_j/h$ then gives the same conditional TV/lifecycle conclusion. The finite-range term remains even when the observed variance is zero. With $R_j = 3584$ bytes, $K = 1$, $\delta = .05$ and $m = 50$, that term alone is approximately 629.6 bytes, larger than the constructed 32-byte mean benefit. This particular bound therefore does not make the example practical at 50 epochs; it is an analytical comparison, not a new deployed or implemented certificate. A tighter externally justified support range or a substantially larger paired benefit can change the calculation. The frozen Hoeffding implementation and all original uncertified outcomes remain unchanged.

A Wasserstein or moment-based allowance would additionally require a declared metric or moment class, an appropriate regularity bound for the packet-cost difference, and independently justified uncertainty for deployment shift. Replacing the TV symbol alone supplies none of that information. Variance sensitivity addresses calibration noise, not missing independence, consumer utility, installation cost or an unsupported drift model.

## S6 Using the full-registry weights in the symmetric optimizer

For declared batch sizes $w_1, \ldots, w_q$ summing to $n$, every node uses

$$r_n(s) = 1 + \sum_a \left(1 - \frac{\binom{n-s}{w_a}}{\binom{n}{w_a}}\right).$$

The registry size remains $n$ inside every subtree state. Treating a subtree as an independently completed registry defines a different law and can change its optimum. Costs are additive across child subtrees; this optimization decomposition does not assume probabilistic independence.

For example, let $n = 4$, $b = 2$, and $(w_1, w_2) = (2, 2)$ at the implemented prices $H = 512, S = 320$. A two-leaf internal node has weight $8/3$, while three- and four-leaf internal nodes have weight 3. A balanced binary tree therefore costs $1152(3 + 2 \cdot 8/3) = 9600$ expected bytes, including construction. A binary chain costs $1152(3 + 3 + 8/3) = 9984$. This is a worked analytical example, not a measured software result.

The integer implementation scales rational weights by a common denominator before comparing costs. Its arithmetic-operation bound is $O(bn^2)$ after weights, with $O(bn)$ cost and split states; integer bit complexity and reconstructing an output tree are additional accounting. Independent enumeration checks 70 small cases and 74,245 feasible labeled tree comparisons. These finite checks test the implementation; the induction in the main article establishes the model-conditional general theorem. The no-learning corollary additionally conditions on the future law given training and selection. Unconditional exchangeability alone is insufficient.

## S7 Evidence units and claim boundaries

The symmetric layout beats frozen full-wave cap32 on all 15 item-shuffled traces and loses on all 15 file-group traces. It costs 21.26–62.13% more on original repeated-order holdouts. With byte observers disabled, the tree service takes 1.61–3.54 times the maintained-counter ledger's operation time. No natural hierarchical consumer demand, general agent speedup, token-billing reduction or physical-network benefit is inferred.

## S8 Reproducing the frozen evidence

The immutable scientific artifact is the public repository commit

`ea874d49362552fd4c1369ef19fc307e6d040c7f`

available at

`https://github.com/xuda1979/future-code/tree/ea874d49362552fd4c1369ef19fc307e6d040c7f/hact-paper/Adaptive_HACT_v5`.

The accompanying source archive contains the current article and supplement. The observation transport in that artifact consists of 53 parts of a 31,138,565-byte lossless ZIP. Its SHA-256 is

Table 2: How the retained evidence supports the article. Re-encoding a trace is not a new upstream execution.

| Evidence | Unit or count | Supported interpretation |
|---|---|---|
| General information separation | 39 exact parameter cases | Implementation conformance for a theorem proved for all admissible fanouts and affine positive-slot prices. |
| Symmetric global reference | 70 cases; 74,245 tree comparisons | Exact optimization under the declared completion law. |
| Public compiler cohort | Six requested projects; five admitted; 25 original admitted runs | A convenience cohort of repeated suites, not independent source changes or agent tasks. Packaging's resource exclusion remains. |
| Controlled dispatch shifts | 36 attempted processes; 30 admitted, including six repaired runs | Six initial VOID source-provenance outcomes are retained. Artificial scheduling interventions do not establish natural drift. |
| Service observer ablation | 240 arms; 10,080 answer checks | Local operation-time and answer conformance for fixed requests; correlated timing blocks do not enlarge the project sample. |
| Retrospective symmetric scoring | The same 25 original and 30 admitted shift runs | A stronger baseline comparison, with all original and file-group losses retained; no additional checker execution. |

`5b003c2566df152e77fef9e0af8f4caace4a446a9fa3bc5e7ce708feda65f35d`

The index binds the original path, size and hash of each of 522 observations. Hydration restores exact bytes, refuses mismatches and is offline. The release inventory independently covers the restored observations as well as their transport parts. Hashes provide byte integrity, not signatures.

With Python 3.12, use numpy 2.3.5, pytest 9.0.2 and pytest-asyncio 1.3.0. From the artifact root:

```
python tools/hydrate_observations.py --require-index
python tools/verify_manifest.py
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 PYTHONPATH=. \
  python -m pytest -q -p pytest_asyncio.plugin
PYTHONPATH=. python -m experiments_v14.exchangeable_study --check
PYTHONPATH=. python -m audit.check_v14
PYTHONPATH=. python -m experiments_v14.report --check
```

The reproduction guide lists all historical and current gates. Those checks remain required: the primary workflow also rechecks v5/v9 arithmetic, v10/v11 separation and exact references, v12 fixed batches and independent follow-ups, v13 adoption and feature span, all manuscript projections, the service oracle, measured query trace, and frozen source/synthetic controls. The retained v12 audit report is `.ai-loop/v12/reports/independent_followups.json`. No missing observation should be repaired by changing its expected hash or regenerating a measured record. A fresh computation goes to a new output path.

# S9 Post-review compiler, transfer and CPU diagnostics

These follow-ups use the internally frozen protocol `docs/V15_REVIEW_PROTOCOL.md`; this is not external preregistration. The freeze binds source and all reused inputs before the new compiler/profile executions. Frozen historical observations and endpoints are unchanged.

## S9.1 Secondary cap64 transfer

Table 3 gives savings of cap64 relative to each named comparator: positive values mean fewer padded bytes. Each range spans three fixed dispatch seeds. The unit remains a scheduled process on the original five admitted projects, with no new checker execution or refitting.

Table 3: Frozen cap64 padded-byte savings (%) on all admitted dispatch traces.

| Project | Policy | vs. historical compact | vs. symmetric |
|---|---|---|---|
| Click | files | 7.78 to 10.13 | 16.09 to 21.01 |
| Jinja | files | 8.90 to 11.58 | 15.50 to 23.64 |
| Pluggy | files | 4.75 to 10.12 | 15.04 to 26.32 |
| attrs | files | 0.00 | 18.26 to 20.48 |
| ExceptionGroup | files | 1.95 to 7.51 | 11.64 to 14.29 |
| Click | items | -1.48 to -0.82 | -10.87 to -9.06 |
| Jinja | items | -1.84 to -1.47 | -10.08 to -8.59 |
| Pluggy | items | -10.97 to -5.26 | -15.29 to -13.46 |
| attrs | items | 0.00 | -9.37 to -8.68 |
| ExceptionGroup | items | -10.08 to -2.38 | -15.53 to -14.83 |

## S9.2 Sparse compact ordering and its resource boundary

The prototype accumulates exact positive pair intersections and singleton counts from bounded-width training waves. It sorts positive Jaccard edges, constructs a maximum-spanning forest, and linearizes each component by deterministic depth-first traversal; zero-affinity components retain canonical ties. This is an established spanning-forest heuristic, not a sparse implementation of average linkage and not an optimizer of the tree objective. For $W$ waves of width at most $w$, at most $W\binom{w}{2}$ positive-pair updates are needed and at most $E \le W\binom{w}{2}$ distinct edges are stored. Counting and ordering take $O(Ww^2 + E\log E + n)$ arithmetic operations and $O(n+E)$ storage apart from integer bit complexity. The compact tree's separate ancestor-scoring index uses $O(n\,\mathrm{height})$ references. An explicit edge limit fails closed rather than dropping observations.

Only original and sparse compact layouts enter this new validation catalogue. Cold construction and touched ancestors are fully charged, with incumbent retention on ties. The complete Packaging replay fits runs 0–1, selects on run 2, then scores runs 3–4. Its two identical held-out completion patterns establish repeatability, not natural transfer. The 2 GiB limit applies to the whole child process, and the peak RSS includes loading, fitting, validation and held-out scoring. The reported fit/validation seconds exclude file loading and hash verification. An independent literal tree traversal verifies every reported held-out price. This experiment neither changes the earlier dense compiler exclusion nor admits Packaging into the 2,048-check service study.

## S9.3 Exclusive CPU accounting

Table 4: Pooled exclusive process-CPU shares (%) for two retained cases per project and arm. cProfile uses `time.process_time`; categories are disjoint and sum to 100 before rounding. H=HACT32; C=counter ledger.

| Project | Arm | Hash | JSON | zlib | Kernel | Tree | Other |
|---|---|---|---|---|---|---|---|
| Click | C | 0.0 | 41.2 | 2.7 | 0.0 | 0.0 | 56.1 |
| Click | H | 1.2 | 19.7 | 0.8 | 4.6 | 48.3 | 25.3 |
| Jinja | C | 0.0 | 40.9 | 2.3 | 0.0 | 0.0 | 56.8 |
| Jinja | H | 1.2 | 19.8 | 0.7 | 4.8 | 47.9 | 25.7 |
| Pluggy | C | 0.0 | 41.5 | 2.6 | 0.0 | 0.0 | 55.9 |
| Pluggy | H | 1.9 | 34.4 | 1.8 | 7.9 | 11.8 | 42.2 |
| attrs | C | 0.0 | 41.1 | 2.6 | 0.0 | 0.0 | 56.3 |
| attrs | H | 0.9 | 15.4 | 0.6 | 3.5 | 58.4 | 21.2 |
| ExceptionGroup | C | 0.0 | 40.3 | 3.7 | 0.0 | 0.0 | 56.0 |
| ExceptionGroup | H | 1.8 | 35.2 | 1.9 | 7.2 | 9.0 | 44.8 |

Input snapshots and the semantic/wire oracle are outside profiling. Every case replays build, every eight-completion ingest and the same cold/warm query sequence as the observer-disabled experiment. Hashing contains the digest wrapper and hash primitives; JSON contains encoding, decoding and canonical serialization wrappers; zlib contains compression/decompression; kernel and tree contain their remaining module-local functions. Thus nested serialization is charged to JSON, not also to a caller's cumulative time. Other includes

orchestration, generic builtins and answer processing. Allocation is distributed across functions and has not been isolated. In Click, the leading HACT functions are `walk`, `to_dict` and `validate_tree`; the retained per-arm records list the twenty largest exclusive functions. Profiler overhead changes the execution regime. These shares locate work; they do not decompose the unprofiled HACT/counter elapsed-time difference or establish the speedup obtainable by removing a category.

## AI assistance and model provenance

OpenAI Codex (October 2026) assisted with this supplement's text, the derivations and proof presentation, the worked example, and the associated code and automated checks. These checks are distinct from human peer review. The authors retain responsibility for the content. Historical experiment manifests do not identify immutable model snapshots; no current model label is retrospectively assigned to those runs.

## References


[1] W. Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. https://doi.org/10.1080/01621459.1963.10500830.

[2] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei. Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control. arXiv:2110.01052v5, 2022. https://arxiv.org/abs/2110.01052v5.

[3] A. Maurer and M. Pontil. Empirical Bernstein Bounds and Sample Variance Penalization. COLT, 2009, Theorem 4 and Corollary 5. https://arxiv.org/abs/0907.3740.